



요구사항 분석

고급모바일프로그래밍 5 주차 | 한기준

학습목표

1. 기능 요구사항과 비기능 요구사항을 구분하고 각각을 작성할 수 있음
2. 사용자 스토리와 유스케이스로 요구사항을 기술할 수 있음
3. 2 주차의 모바일 제약을 검증 가능한 비기능 요구사항으로 번역할 수 있음

학습내용

1. 요구사항의 종류와 계층
2. 사용자 스토리 · 유스케이스 · 비기능 요구사항 작성법
3. 검증 가능한 문장 쓰기와 제안서 작성

오늘의 흐름

총 40 분 · 이론 → 실제 서비스 사례 → 우리 프로젝트 적용 순서로 진행함

구간	분량	내용
도입	4 분	왜 요구사항 분석이 필요한가 · 사례 : 구글 웨이브
요구사항의 종류	5 분	기능 / 비기능 , 요구사항 계층 · 사례 : 메신저 읽음 표시
사용자 스토리	8 분	스토리 형식 , INVEST, 인수 조건 · 사례 : 배달 앱 주문 알림
유스케이스	6 분	액터와 시나리오 , 대안 흐름 , 무엇을 언제 쓰는가
모바일 비기능	9 분	제약을 요구사항으로 번역 · 사례 : 지도 · 음악 앱 오프라인
검증과 추적	5 분	모호한 표현 제거 , 추적 매트릭스
제안서 작성	3 분	제안서 목차와 평가 기준 매핑 , 이번 주 할 일

2 주차에서 본 제약은 결국 요구사항 문서 위에서 결정됨 .

제약	2 주차에서 한 말	5 주차에서 할 일
에너지	위치 수집 주기는 요구사항 단계에서 정해야 함	갱신 주기와 정확도를 수치로 명세
메모리	사용자 입력은 즉시 저장해야 함	어떤 데이터를 언제 저장하는지 명세
네트워크	연결 없이 되어야 할 최소 기능을 정의해야 함	기능별 온라인 / 오프라인 구분표
응답성	모든 터치에 100ms 안에 피드백	화면별 응답 시간 목표치
신뢰 경계	검증은 서버에서 다시 해야 함	검증 위치와 권한 요구사항 명세

오늘은 ' 무엇을 만들지 ' 를 말이 아니라 문서로 합의하는 주차임 .

요구사항 분석은 세 가지를 동시에 함 .

합의

팀원 네 명이 같은 것을 상상하고 있는지
맞춰 봄 . 머릿속에만 있으면 반드시
어긋남 .

경계

무엇을 만들지보다 무엇을 만들지
않을지를 못박음 . 범위가 새는 것을 막는
장치임 .

기준

나중에 ' 다 만들었다 ' 를 판정할 근거가
됨 . 기준이 없으면 완성 여부를 말할 수
없음 .

세 번째가 특히 중요함 . 15 주 프로젝트에서 ' 어디까지가 완성인가 ' 를 미리 정해 두지 않으면 , 학기 말에 팀마다 기준이 달라짐 .

같은 말을 들어도 네 사람이 네 가지를 상상함 .

팀 회의에서 나온 한마디 : " 사용자가 재료를 편하게 등록할 수 있게 하자 "

팀원 A

영수증을 찍으면
자동으로 다 들어가는 것

팀원 B

바코드를 찍으면
품명이 뜨는 것

팀원 C

목록에서 골라
체크만 하는 것

팀원 D

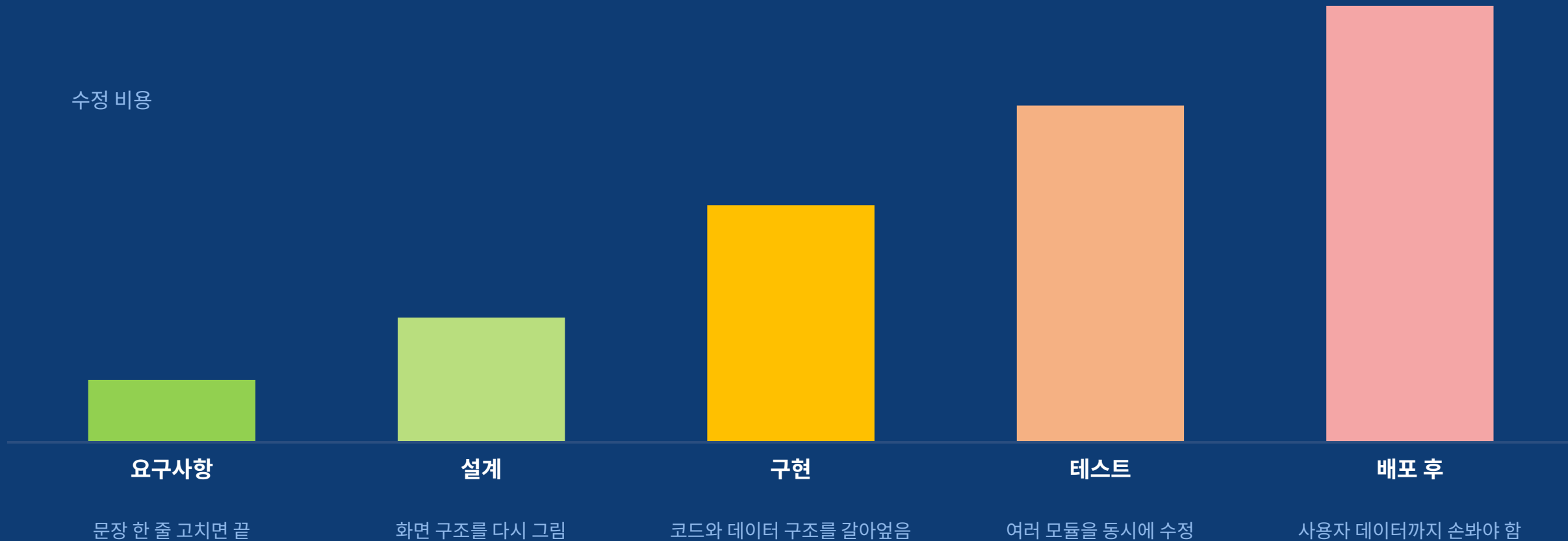
음성으로 말하면
입력되는 것

네 명이 각자 작업을 시작하면

A 는 OCR 서버를 알아보고 , B 는 바코드 라이브러리를 붙이고 , C 는 데이터베이스를 채우고 , D 는 음성 인식을 검토함 . 2 주가 지나서야 서로 다른 것을 만들고 있었다는 사실이 드러남 . 요구사항 문서는 이 2 주를 아끼기 위한 장치임 .

요구사항 결함은 늦게 발견될수록 비싸짐 .

수정 비용



비율은 개념적 표현임 . 요점은 ' 분석에 쓰는 한 시간이 구현에서 하루를 아낀다 ' 는 것 .

CASE STUDY

구글 웨이브 — 기술이 아니라 문제 정의에서 무너진 사례

무슨 일이 있었나

- 2009 년 구글이 이메일 · 메신저 · 문서 협업을 하나로 합친 서비스로 발표함
- 실시간 공동 편집 등 당시로는 앞선 기술을 담고 있었음
- 초대장이 거래될 만큼 기대가 컸음
- 그러나 1 년여 만인 2010 년 개발 중단이 발표됨

요구사항 관점에서

- ' 누가 어떤 상황에서 무엇이 불편한가 ' 가 끝까지 분명하지 않았음
- 메일인지 채팅인지 문서인지 사용자가 판단하지 못했음
- 기능은 많았지만 ' 이걸 왜 쓰는가 ' 에 한 문장으로 답할 수 없었음
- 기술적 완성도가 문제 정의의 부재를 메워 주지 못함

우리 프로젝트에 주는 시사점

제안서 첫 줄에 ' 누가 , 어떤 상황에서 , 무엇이 불편한가 ' 를 한 문장으로 쓰지 못한다면 , 그 뒤에 적는 기능 목록은 근거가 없는 것임 . 기술 스택은 나중에 바꿀 수 있지만 문제 정의는 바꾸면 프로젝트가 처음으로 돌아감 .

좋은 요구사항은 네 가지 조건을 만족함 .

01 명확성

읽는 사람마다 다르게 해석되지 않음

02 검증 가능성

만족했는지 여부를 판정할 수 있음

03 추적 가능성

어떤 화면 · 어떤 테스트와 연결되는지 알 수 있음

04 실현 가능성

15 주 안에 4 인 팀이 실제로 만들 수 있음

오늘 남은 시간은 이 네 조건을 만족하는 문장을 쓰는 방법을 다룸 .

요구사항은 머리에서 짜내는 것이 아니라 모아 오는 것임 .

인터뷰

대상 사용자 3~5 명에게 직접 물어봄

" 마지막으로 재료를 버린 게 언제였나요 ?" — 의견이 아니라 경험을 물을 것

관찰

실제로 하는 모습을 지켜봄

말로는 ' 냉장고를 잘 관리한다 ' 고 해도 실제 행동은 다른 경우가 많음

경쟁 앱 분석

같은 문제를 다루는 앱을 직접 써 봄

어떤 기능이 있는지보다 ' 왜 그렇게 만들었는지 ' 를 볼 것

리뷰 읽기

스토어 리뷰의 불만을 읽음

★1~2 점 리뷰에 요구사항이 가장 많이 들어 있음

4 인 팀이면 각자 한 가지씩 맡아 하루면 끝남 . 이 과정 없이 쓴 요구사항은 대부분 추측임 .

기능 요구사항은 무엇을 하는가, 비기능 요구사항은 얼마나 잘 하는가.

기능 요구사항

- 사용자는 사진을 촬영해 게시물을 등록할 수 있음
- 사용자는 반경 3km 이내의 게시물만 조회할 수 있음
- 작성자는 자신의 게시물을 삭제할 수 있음

동사로 끝남 — '~ 할 수 있음'

비기능 요구사항

- 게시물 목록은 1 초 이내에 첫 화면을 표시함
- 오프라인에서도 저장한 게시물을 열람할 수 있음
- 위치 권한은 지도 화면에 진입할 때만 요청함

수치 · 조건으로 끝남 — 측정 가능해야 함

매 학기 반복되는 문제 — 제안서에 기능 요구사항만 20 줄 적고 비기능 요구사항이 한 줄도 없음. 그러면 8~9 주차 기술 선정의 근거가 사라짐.

비기능 요구사항에는 여섯 가지 범주가 있음 .

성능

응답 시간 , 처리량 , 자원 사용량

가용성

오프라인 동작 , 장애 시 복구

보안

인증 , 권한 , 데이터 보호

사용성

학습 용이성 , 접근성 , 오류 복구

호환성

지원 OS 버전 , 화면 크기

유지보수성

로그 , 설정 분리 , 테스트 가능성

노란 테두리 세 가지는 이번 학기에 반드시 적을 것

성능 · 가용성 · 보안은 2 주차에서 본 제약과 그대로 맞물림 . 나머지 세 가지는 여유가 있을 때 추가하면 됨 .

기술 선택을 결정하는 것은 기능이 아니라 비기능 요구사항임 .

비기능 요구사항	그래서 무엇이 결정되는가	관련 주차
오프라인에서 읽고 쓸 수 있어야 함	로컬 DB 가 필수가 되고 , 동기화 · 충돌 해결 설계가 추가됨	11 주차
알림이 즉시 도착해야 함	푸시를 지원하는 백엔드로 선택지가 좁혀짐	10 주차
여러 사용자가 동시에 같은 화면을 봐야 함	실시간 DB 또는 소켓 서버가 필요해짐	10 주차
배터리를 과도하게 쓰지 않아야 함	위치 갱신 주기와 백그라운드 동작 방식이 제한됨	10 주차
개인정보를 다룸	서버 검증과 권한 설계가 선택이 아니라 전제가 됨	13 주차

8 주차 크로스 플랫폼 조사와 10 주차 구현 기술 확정 보고서는 이 표의 왼쪽 열에서 출발함 .

요구사항은 세 층으로 나뉨 .

비즈니스 요구사항

왜 이 앱을 만드는가

예) 자취생이 식재료를 버리지 않게 한다

사용자 요구사항

사용자가 무엇을 하려는가

예) 유통기한이 임박한 재료를 미리 알고 싶다

시스템 요구사항

시스템이 무엇을 제공하는가

예) 유통기한 D-3 시점에 푸시 알림을 전송한다

위에서 아래로 내려가며 구체화됨 . 시스템 요구사항만 있고 위 두 층이 없으면 '왜 이 기능인가'에 답할 수 없음 .

CASE STUDY

카카오톡 읽음 표시 — 기능 한 줄이 만드는 파급

기능 요구사항 한 줄 : " 보낸 메시지를 상대가 읽었는지 확인할 수 있음 "

실시간성

읽음 상태가 몇 초 안에 반영되어야 함 → 폴링으로는 부족, 푸시 또는 소켓 필요

데이터 모델

메시지마다 수신자별 읽음 상태를 저장해야 함 → 단체방이면 N 배로 늘어남

오프라인

오프라인에서 읽은 경우 언제 서버에 반영할지 정해야 함

프라이버시

읽고도 안 읽은 척하고 싶은 사용자가 있음 → 끄기 옵션 제공 여부가 정책 결정이 됨

기능 요구사항을 적을 때마다 " 이게 비기능 쪽에 무엇을 요구하는가 " 를 한 번씩 물어볼 것 .

사용자 스토리는 세 조각으로 씀 .

누가

As a

자취 3 년 차 1 인 가구로서

무엇을

I want

유통기한이 임박한 재료를 알림으로 받고 싶음

왜

So that

재료를 버리지 않고 쓸 수 있기 때문

가장 자주 빠지는 조각

세 번째 'So that' 임 . 이유를 쓰지 못하는 기능은 대부분 없어도 되는 기능이며 , 쓰는 순간 더 간단한 대안이 보이는 경우가 많음 .

같은 기능도 쓰는 방식에 따라 쓸모가 달라짐 .

이렇게 쓰면 쓸모가 없음

- " 푸시 알림 기능을 구현한다 "
- " 사용자 편의성을 높인다 "
- "AI 로 추천해 준다 "
- "UI 를 직관적으로 만든다 "

시스템 입장에서 쓴 문장 — 누가 왜 쓰는지 알 수 없음

이렇게 쓰면 쓸모가 있음

- " 유통기한 3 일 전에 알림을 받고 싶음 — 버리지 않으려고 "
- " 장을 보기 전에 남은 재료를 확인하고 싶음 — 중복 구매를 피하려고 "
- " 남은 재료로 만들 수 있는 요리를 보고 싶음 — 메뉴를 고민하지 않으려고 "

사용자 입장에서 쓴 문장 — 검증할 수 있음

왼쪽 문장들은 전부 " 그래서 다 만들었는지 어떻게 아는가 " 에 답할 수 없음 .

쓸만한 스토리인지 INVEST 로 점검함 .



Independent

다른 스토리를 끝내야만 할 수 있는 상태가 아님



Valuable

사용자에게 의미가 있음 — 내부 작업은 스토리가 아님



Small

한 사람이 며칠 안에 끝낼 크기임



Negotiable

구현 방법까지 못박지 않음 — 어떻게는 열어 둬



Estimable

얼마나 걸릴지 가늠할 수 있을 만큼 구체적임



Testable

완료 여부를 판정할 방법이 있음

4 인 팀 기준으로 S 가 가장 자주 깨짐 . ' 로그인 기능 ' 은 스토리가 아니라 에픽임 .

모든 스토리를 같은 무게로 두면 범위를 닫을 수 없음 .

Must

없으면 앱이 성립하지 않음

2~3 개로 제한할 것

Should

있어야 하지만 없어도 동작함

시간이 남으면 구현

Could

있으면 좋음

대부분 이번 학기에 못 함

Won't

이번 학기에 하지 않기로 한 것

제안서에 명시할 것

Must 가 다섯 개를 넘으면 범위가 과한 것임 . 네 번째 칸을 비워 두는 팀이 가장 많이 흔들림 .

스토리 하나에 **인수 조건**을 붙이면 비로소 검증이 가능해짐 .

스토리 : " 유통기한이 임박한 재료를 알림으로 받고 싶음 "

Given 냉장고에 유통기한이 3 일 남은 재료가 등록되어 있을 때

When 매일 오전 9 시가 되면

Then 해당 재료 이름을 포함한 푸시 알림을 1 건 전송함

And 알림을 탭하면 해당 재료의 상세 화면으로 이동함

이 형식의 장점 — 그대로 테스트 케이스가 됨 . 14 주차 테스트 프레임워크 강의에서 이 문장들이 다시 등장함 .

CASE STUDY

배달의민족 주문 상태 알림 — 에픽을 스토리로 쪼개기

에픽 : " 주문한 음식이 어디쯤 왔는지 알고 싶음 "

S1 주문이 가게에 접수되면 알림을 받고 싶음 — 주문이 들어갔는지 불안하지 않으려고

S2 조리가 시작되면 알림을 받고 싶음 — 언제쯤 나갈지 가능하려고

S3 배달이 시작되면 알림과 함께 예상 도착 시각을 보고 싶음 — 미리 준비하려고

S4 앱을 열면 현재 상태를 한 화면에서 보고 싶음 — 알림을 놓쳤을 때를 대비해

쪼개기 뒤에 보이는 것

S1~S3 은 알림 하나씩이라 비슷해 보이지만 , S4 는 화면이 하나 더 필요함 . 쪼개기 전에는 ' 알림 기능 ' 이라는 한 덩어리로 보여서 작업량을 가늠할 수 없었음 . 4 인 팀의 분담도 쪼개기 뒤에야 가능해짐 .

사용자 스토리와 유스케이스는 쓰임새가 다름 .

사용자 스토리		유스케이스
분량	한두 문장	한 페이지 내외의 명세
목적	대화를 시작하기 위한 메모	흐름을 빠짐없이 기술하기 위한 문서
강점	빠르게 쓰고 쉽게 버릴 수 있음	예외 · 대안 흐름까지 드러남
약점	복잡한 흐름을 담기 어려움	쓰는 데 시간이 많이 듦
적합한 경우	기능이 단순하고 자주 바뀔 때	결제 · 인증처럼 실패 경로가 중요할 때

둘 다 쓸 필요는 없음 . 대부분의 기능은 스토리로 충분하고 , 흐름이 복잡한 한두 개만 유스케이스로 쓰면 됨 .

액터는 사람만이 아님 .

주 액터

목표를 달성하려고 시스템을 사용하는 쪽

앱 사용자 , 관리자

보조 액터

시스템이 목표를 달성하려고 호출하는 쪽

OCR 서버 , 결제 대행사 , 지도 API

시간

특정 시각이 되면 동작을 유발하는 것

매일 오전 9 시 알림 스케줄러

외부 이벤트

시스템 밖에서 발생해 흐름을 바꾸는 것

네트워크 연결 복구 , 권한 변경

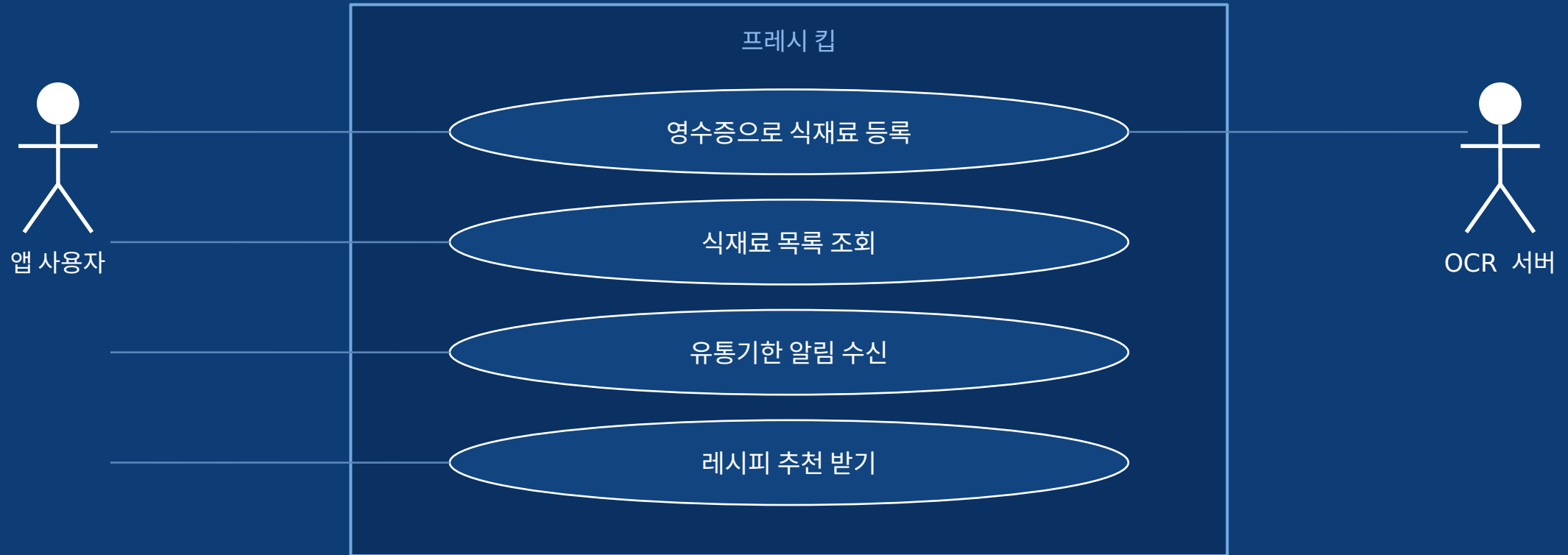
세 번째와 네 번째를 빠뜨리면 '알림이 언제 오는가', '연결이 끊기면 어떻게 되는가'가 요구사항에서 누락됨 .

유스케이스의 값은 정상 흐름이 아니라 대안 흐름에 있음 .

항목	내용
유스케이스명	영수증으로 식재료 등록
액터	앱 사용자 (주 액터), OCR 서버 (보조 액터)
사전 조건	로그인 상태이며 카메라 권한이 허용되어 있음
정상 흐름	① 촬영 버튼을 누름 ② 영수증을 촬영함 ③ 서버가 품목을 인식함 ④ 인식 결과를 확인 · 수정함 ⑤ 저장함
대안 흐름	3a. 인식 실패 → 수동 입력 화면으로 전환 / 3b. 네트워크 단절 → 큐에 저장 후 재시도 / 4a. 오인식 → 사용자가 직접 수정
사후 조건	등록된 재료가 목록과 로컬 DB 에 반영되어 있음

학생 제안서는 보통 정상 흐름만 적음 . 실제 개발 시간은 대부분 대안 흐름에서 쓰임 .

다이어그램은 '누가 무엇을 할 수 있는가' 만 보여 줌 .



다이어그램은 범위를 한눈에 보여 주는 용도임 . 세부 흐름은 앞 장의 명세로 적어야 함 .

2 주차의 제약을 측정 가능한 문장으로 바꾸면 비기능 요구사항이 됨 .

제약	막연한 표현 (쓰면 안 됨)	측정 가능한 요구사항 (이렇게 쓸 것)
에너지	배터리를 적게 쓴다	위치 추적 중 갱신 주기는 10 초 , 정확도는 50m 수준으로 요청함
메모리	데이터가 날아가지 않게 한다	작성 중인 글은 화면 이탈 시 로컬에 즉시 저장하고 재진입 시 복원함
네트워크	오프라인을 지원한다	저장한 자료 목록은 네트워크 없이 조회할 수 있고 , 등록은 큐에 보관 후 재전송함
응답성	빠르게 반응한다	모든 버튼은 100ms 이내에 시각 피드백을 표시하고 , 목록 첫 화면은 1 초 이내에 표시함
신뢰 경계	보안을 고려한다	위치 권한은 지도 진입 시에만 요청하고 , 거절 시 수동 입력 경로를 제공함

오른쪽 열의 공통점 — 숫자 또는 조건이 들어 있어 만족 여부를 판정할 수 있음 .

오프라인 지원은 기능별로 나누어 표로 정의할 것 .

기능	온라인	오프라인	복구 시 동작
재료 목록 조회	서버 최신 데이터	로컬에 저장된 마지막 데이터	자동 새로고침
재료 직접 등록	즉시 반영	로컬 저장 후 대기	큐에 쌓인 순서대로 전송
영수증 촬영 등록	즉시 인식	사진만 저장 , 인식 보류	연결 시 인식 요청
유통기한 알림	푸시 수신	로컬 알림으로 대체	서버 알림과 중복 제거
레시피 추천	사용 가능	사용 불가 — 안내 문구 표시	자동 재시도 없음

이 표를 요구사항 분석서에 그대로 넣을 것

맨 오른쪽 열이 11 주차 ' 오프라인 퍼스트와 데이터 동기화 ' 에서 다시 등장함 . 지금 비워 두면 그때 설계를 처음부터 다시 하게 됨 .

데이터가 어디서 오는지 적지 않으면 구현 단계에서 막힘 .

데이터	출처	갱신 주기	없을 때 대안
식재료 품목명	사용자 입력 + 영수증 인식	등록 시점	수동 입력 화면 제공
유통기한	사용자 입력 (기본값 제안)	등록 시점	품목별 평균값으로 제안
레시피	공공 데이터 또는 자체 수집	학기 중 1 회 구축	재료 조합 규칙으로 대체
사용자 위치	기기 GPS	지도 화면 진입 시	지역 직접 선택

맨 오른쪽 열을 반드시 채울 것

매 학기 'AI 가 추천해 준다 ' 는 주제가 나오는데 , 추천에 쓸 데이터가 어디서 오는지 적지 못해 11 주차에 멈춤 .
데이터를 구할 수 없다면 지금 주제를 바꾸는 편이 훨씬 낫 .

권한과 응답 시간도 요구사항으로 적을 것 .

권한 요구사항

- 어떤 권한이 필요한가 (카메라 · 위치 · 알림 · 저장소)
- 언제 요청하는가 (앱 시작 시 ✕ / 기능 진입 시 ○)
- 왜 필요한지 어떻게 설명하는가
- 거절당하면 어떤 경로로 대체하는가

응답 시간 요구사항

100ms	모든 터치에 시각 피드백
1 초	화면 전환 , 목록 첫 표시
3 초	영수증 인식 결과 표시
그 이상	진행률과 취소 수단 제공

2 주차에서 본 인간 지각 임계값 (100ms / 1 초 / 10 초) 이 그대로 요구사항 수치가 됨 .

CASE STUDY

구글 지도 · 스포티파이 — 오프라인 요구사항을 문장으로 복원하면

구글 지도

- 사용자는 지역을 선택해 지도 데이터를 미리 내려받을 수 있음
- 내려받은 지역 안에서는 네트워크 없이 검색과 경로 안내를 사용할 수 있음
- 실시간 교통 정보는 오프라인에서 제공하지 않으며, 그 사실을 화면에 표시함

스포티파이

- 사용자는 재생 목록 단위로 음원을 내려받을 수 있음
- 내려받은 음원은 네트워크 없이 재생할 수 있음
- 검색과 신규 음원 추가는 오프라인에서 제공하지 않음

두 사례의 공통 구조

① 오프라인에서 되는 것을 명시함 ② 안 되는 것도 명시함 ③ 안 된다는 사실을 사용자에게 알리는 방법까지 적음 .
세 번째가 빠지면 사용자는 앱이 고장났다고 판단함 . 요구사항 문장에 ' 안내 문구를 표시함 ' 까지 넣어야 완성임 .

검증할 수 없는 단어는 **요구사항이 아니라 희망사항임**.

쓰면 안 되는 표현	왜 안 되는가	이렇게 바꿀 것
빠르게 / 신속하게	얼마나 빠르지 판정 불가	1 초 이내에 첫 화면을 표시함
사용하기 쉽게	쉬움의 기준이 사람마다 다름	주요 기능까지 3 번 이내의 탭으로 도달함
안정적으로	무엇이 안정인지 불명확	네트워크 단절 시 작성 중인 데이터를 잃지 않음
적절히 / 필요시	판단 주체가 없음	유통기한 3 일 전 오전 9 시에 1 회 전송함
등 / 기타	범위가 무한히 열림	항목을 모두 나열하거나 범위를 명시함

자기 문서에서 위 단어를 찾아 지우는 것만으로도 분석서 품질이 크게 올라감.

요구사항은 화면과 테스트에 연결될 때 추적 가능해짐 .

ID	요구사항	관련 화면	검증 방법	우선순위
FR-01	영수증을 촬영해 재료를 등록할 수 있음	촬영 , 인식 결과 확인	촬영 → 저장 후 목록에 표시되는지	필수
FR-02	재료를 수동으로 등록할 수 있음	수동 입력	입력 → 목록 반영 확인	필수
FR-03	남은 재료로 레시피를 추천받을 수 있음	레시피 목록	재료 3 종 등록 후 결과 유무 확인	선택
NFR-01	목록 첫 화면을 1 초 이내에 표시함	재료 목록	저사양 기기에서 측정	필수
NFR-02	오프라인에서 재료 목록을 조회할 수 있음	재료 목록	기내 모드에서 진입 확인	필수

맨 오른쪽 ' 우선순위 ' 열이 범위 관리의 핵심임 . 11~13 주차에 시간이 모자라면 ' 선택 ' 부터 덜어내면 되고 , 그 판단 근거가 분석서에 이미 적혀 있게 됨 .

요구사항은 바뀔 . 바뀌는 것 자체는 문제가 아님 .

문제가 되는 변경

기록 없이 말로만 바뀔
누가 언제 왜 바꿨는지 모름
팀원마다 다른 버전을 기억함

괜찮은 변경

문서에 반영하고 날짜를 남김
영향받는 화면과 테스트를 같이 확인함
범위가 늘면 무엇을 뺄지 함께 정함

GitHub 를 쓰고 있다는 이점

4 주차에 실습한 GitHub 가 여기서 쓰임 . 요구사항 문서를 저장소에 함께 두면 변경 이력이 자동으로 남고 , 이슈로 변경을 논의한 기록도 남음 .
문서를 카톡으로 주고받으면 이 이점이 사라짐 .

쓴 사람 말고 다른 팀원이 읽어 보는 것만으로 절반이 걸러짐 .

☐ 이 문장을 읽고 나는 같은 화면을 상상했는가

☐ 만족 여부를 어떻게 판정할지 말할 수 있는가

☐ '빠르게 · 적절히 · 등' 같은 단어가 남아 있는가

☐ 이 요구사항이 어떤 화면과 연결되는지 알 수 있는가

☐ 구현 방법까지 못박아 버리지는 않았는가

☐ 우선순위가 Must 인데 정말 없으면 안 되는가

대면 수업 시간에 팀을 바꿔 서로의 문서를 읽는 방식을 권함 . 자기 문서의 빈틈은 자기 눈에 거의 보이지 않음 .

요구사항 분석서는 **제안서를 확장한 문서임** (7 주차 10.15 목 12 시 제출).

장	내용	오늘 다룬 것
1. 개요	문제 정의, 대상 사용자, 범위	제안서에서 가져옴
2. 기능 요구사항	사용자 스토리 + 인수 조건, 우선순위 (MoSCoW)	사용자 스토리 절
3. 주요 흐름	복잡한 기능 1~2 개의 유스케이스 명세	유스케이스 절
4. 비기능 요구사항	성능 · 가용성 · 보안, 오프라인 동작 정의표	모바일 비기능 절
5. 데이터	데이터 출처와 갱신 주기	데이터 요구사항
6. 추적 매트릭스	요구사항 ↔ 화면 ↔ 검증 방법	검증과 추적 절

제안서 (10.08) 를 제대로 쓰면 분석서 (10.15) 는 2 · 4 · 5 장을 채우는 작업만 남음 . 두 문서는 이어서 쓰는 하나의 흐름임 .

제안서는 오늘 배운 것을 순서대로 적으면 됨 .

제안서 항목	오늘 다룬 내용	분량 기준
문제 정의	누가 , 어떤 상황에서 , 무엇이 불편한가 — 한 문장	3~5 줄
대상 사용자	비즈니스 · 사용자 요구사항 계층	3~5 줄
모바일 필연성	어떤 모바일 역량이 핵심 메커니즘인가 (2 주차)	3~5 줄
핵심 기능	사용자 스토리 형식으로 3~5 개	반 페이지
비기능 요구사항	오프라인 · 응답 시간 · 권한 중 최소 3 개	반 페이지
하지 않을 것	이번 학기에 다루지 않을 범위를 명시	3 줄
가장 위험한 가정	틀리면 프로젝트가 무너지는 전제 하나	2~3 줄

과제 20 점 중 제안서가 10 점임 . 평가 기준은 ' 필요한 내용만 갖추고 있으면 만점 ' — 분량보다 항목의 완결성이 중요함 .

매 학기 반복되는 감점 사유 .

문제 정의가 기능 설명임

"냉장고 관리 앱을 만든다" 는 문제 정의가 아니라 해결책임 . 불편한 상황이 먼저 와야 함 .

대상이 ' 모든 사람 ' 임

범위가 넓을수록 요구사항이 흐려짐 . 좁힐수록 쓸 문장이 분명해짐 .

비기능 요구사항이 없음

기능만 20 줄 . 기술 선정의 근거가 사라지고 10 주차 보고서에서 막힘 .

검증 불가능한 표현

' 빠르게 ' , ' 직관적으로 ' , ' 적절히 ' . 평가할 방법이 없음 .

' 하지 않을 것 ' 이 비어 있음

범위를 닫지 않으면 학기 중반에 기능이 계속 늘어남 .

이번 주 해야 할 일

시기	구분	할 일
화 ~ 수	팀 활동	주제 확정 , 문제 정의와 대상 사용자 문장 작성
목 (10.01)	대면 수업	주제 확정 발표 , 요구사항 분석 착수 — 스토리 초안 작성
다음 주	팀 활동	비기능 요구사항과 '하지 않을 것' 정리 , 제안서 마무리
10.08 (목) 12 시	제출	프로젝트 제안서 · 과제 10 점

다음 주차 예고

6 주차 — GitHub Actions. 요구사항이 테스트로 이어지는 통로를 다룸 . 오늘 쓴 인수 조건 (Given/When/Then) 이 자동 검증으로 연결되는 지점임 .

요구사항은 ' 무엇을 만들까 ' 를 적는 문서가 아니라 ,
' 다 만들었는지 어떻게 알까 ' 를 미리 적어 두는 문서임 .

1 두 종류를 모두 쓸 것

기능 요구사항만 적으면 기술 선정의
근거가 사라짐 . 비기능이 아키텍처를 정함 .

2 측정 가능하게 쓸 것

' 빠르게 ' 가 아니라 ' 1 초 이내 ' .
판정할 수 없으면 요구사항이 아님 .

3 범위를 닫을 것

' 하지 않을 것 ' 을 명시해야
학기 중반에 기능이 새지 않음 .

프로젝트 제안서 제출 · 10 월 8 일 (목) 12 시 · keejun.han@hansung.ac.kr